UD2.1: Listas y Bucles

Módulo: Desarrollo de Aplicaciones con Lenguaje de Programación (DALP)
Primera parte: Estructuras de datos básicas e iteración

Índice de contenidos

1. Introducción a las estructuras de datos

- 1.1. ¿Qué son las estructuras de datos?
- 1.2. Tipos de datos simples vs. complejos
- 1.3. Mutabilidad e inmutabilidad

2. Listas

- 2.1. Concepto y sintaxis
- 2.2. Creación y acceso a elementos
- 2.3. Índices positivos y negativos
- 2.4. Métodos principales de listas
  - append(), extend(), insert()
  - remove(), pop(), clear()
  - sort(), reverse(), copy()
  - count(), index()
- 2.5. Operaciones con listas
  - Concatenación y repetición
  - Pertenencia (in, not in)
  - Longitud (len())
- 2.6. Listas anidadas

3. Bucle while

- 3.1. Concepto y sintaxis
- 3.2. Condiciones de continuación y terminación
- 3.3. Bucles infinitos y cómo evitarlos

- 3.4. Comparación while vs. for

4. Bucle for

- 4.1. Concepto y sintaxis
- 4.2. Iteración sobre listas
- 4.3. La función range()
  - range(n)
  - range(inicio, fin)
  - range(inicio, fin, paso)

5. Conceptos avanzados de bucles

- 5.1. Control de flujo en bucles
  - break: salir del bucle
  - continue: saltar a la siguiente iteración
  - pass: marcador de posición
- 5.2. Bucles con else
  - while con else
  - for con else
- 5.3. Bucles anidados
  - Bucles for anidados
  - Bucles while anidados
  - Combinaciones while-for

6. Ejercicios prácticos

- 6.1. Ejercicios de listas
- 6.2. Ejercicios de while
- 6.3. Ejercicios de for
- 6.4. Ejercicios integradores

> **Nota**: Este documento cubre la primera parte de UD2. Las secciones sobre Tuplas, Conjuntos, Diccionarios y herramientas avanzadas se cubrirán en UD2.2.

1. Introducción a las estructuras de datos

1.1. ¿Qué son las estructuras de datos?

Una **estructura de datos** es una forma organizada de almacenar, gestionar y acceder a múltiples valores relacionados. Hasta ahora hemos trabajado con variables que almacenan un único valor:

```
nombre = "Ana"
edad = 25
precio = 19.99
aprobado = True
```

Sin embargo, en programas reales necesitamos manejar colecciones de datos: una lista de estudiantes, los precios de varios productos, las temperaturas de una semana, etc. Las estructuras de datos nos permiten agrupar estos valores de forma eficiente.

Ventajas de usar estructuras de datos:

- Organizar información relacionada bajo un único identificador
- Procesar múltiples valores con bucles
- Modelar problemas del mundo real de forma natural
- Reducir la cantidad de variables individuales

Ejemplo comparativo:

```
# Sin estructura de datos (ineficiente)
estudiante1 = "Ana"
estudiante2 = "Carlos"
estudiante3 = "María"
estudiante4 = "Juan"

# Con estructura de datos (eficiente)
estudiantes = ["Ana", "Carlos", "María", "Juan"]
```

1.2. Tipos de datos simples vs. complejos

Python clasifica los tipos de datos en dos categorías principales:

Tipos de datos simples (escalares)

Almacenan un único valor atómico:

Tipo	Descripción	Ejemplo
`int`	Números enteros	`42` , `-17` , `0`
`float`	Números decimales	`3.14` , `-0.5` , `2.0`
`str`	Cadenas de texto	`"Hola"` , `'Python'`
`bool`	Valores lógicos	`True` , `False`
`NoneType`	Ausencia de valor	`None`

```
# Ejemplos de tipos simples
contador: int = 10
temperatura: float = 23.5
mensaje: str = "Bienvenido"
activo: bool = True
resultado = None
```

Tipos de datos complejos (colecciones)

Almacenan múltiples valores en una única estructura:

Tipo	Descripción	Mutable	Ordenado	Duplicados
lista (`list`)	Secuencia ordenada	Sí	Sí	Sí
tupla (`tuple`)	Secuencia inmutable	No	Sí	Sí
conjunto (`set`)	Colección sin orden	Sí	No	No
diccionario (`dict`)	Pares clave-valor	Sí	Sí*	No (claves)

*Desde Python 3.7, los diccionarios mantienen el orden de inserción.

```
# Ejemplos de tipos complejos
numeros: list = [1, 2, 3, 4, 5]
coordenadas: tuple = (10.5, 20.3)
colores: set = {"rojo", "verde", "azul"}
estudiante: dict = {"nombre": "Ana", "edad": 25}
```

1.3. Mutabilidad e inmutabilidad

La **mutabilidad** determina si un objeto puede modificarse después de su creación.

Tipos inmutables

No se pueden modificar una vez creados. Cualquier operación que parezca modificarlos en realidad crea un nuevo objeto:

- `int` , `float` , `bool` , `str` , `tuple` , `frozenset`

```
# Ejemplo con strings (inmutables)
texto = "Hola"
print(id(texto))  # 140234567890 (dirección de memoria)

texto = texto + " Mundo"
print(id(texto))  # 140234567912 (diferente dirección, nuevo objeto)

# Intentar modificar un carácter individual produce error
# texto[0] = "h"  # TypeError: 'str' object does not support item assignment


# Ejemplo con tuplas (inmutables)
coordenadas = (10, 20)
# coordenadas[0] = 15  # TypeError: 'tuple' object does not support item assignment
```

Tipos mutables

Pueden modificarse después de su creación sin cambiar su identidad en memoria:

- `list` , `set` , `dict`

```
# Ejemplo con listas (mutables)
numeros = [1, 2, 3]
print(id(numeros))  # 140234568000

numeros.append(4)  # Modifica la lista in-place
print(id(numeros))  # 140234568000 (misma dirección)
print(numeros)      # [1, 2, 3, 4]

numeros[0] = 10     # Modifica un elemento
print(numeros)      # [10, 2, 3, 4]
```

¿Por qué es importante la mutabilidad?

1. **Rendimiento**: Modificar un objeto mutable es más eficiente que crear uno nuevo
2. **Efectos secundarios**: Las funciones pueden modificar argumentos mutables
3. **Uso como claves**: Solo los inmutables pueden ser claves de diccionarios
4. **Seguridad**: Los inmutables son más seguros en entornos concurrentes

```
# Ejemplo de efecto secundario con mutables
def agregar_elemento(lista, elemento):
    lista.append(elemento)  # Modifica la lista original

mis_numeros = [1, 2, 3]
agregar_elemento(mis_numeros, 4)
print(mis_numeros)  # [1, 2, 3, 4] - la lista original cambió


# Ejemplo de copia vs. referencia
# Con mutables, asignar crea una referencia, no una copia
lista_a = [1, 2, 3]
lista_b = lista_a       # lista_b apunta al mismo objeto
lista_b.append(4)
print(lista_a)          # [1, 2, 3, 4] - también cambió

# Para copiar, usar .copy() o slicing
lista_c = lista_a.copy()
lista_c.append(5)
print(lista_a)          # [1, 2, 3, 4] - no cambió
print(lista_c)          # [1, 2, 3, 4, 5]
```

Tabla resumen

Característica	Inmutable	Mutable
Se puede modificar	No	Sí
Crea nuevo objeto al cambiar	Sí	No
Puede ser clave de diccionario	Sí	No
Más eficiente al modificar	No	Sí
Más seguro (sin efectos secundarios)	Sí	No

Característica	Inmutable	Mutable
Ejemplos	`str` , `tuple` , `int`	`list` , `dict` , `set`

Regla práctica: Si necesita cambiar frecuentemente, use mutable (`list`). Si los datos no deben cambiar o necesita usarlos como clave, use inmutable (`tuple`).

2. Listas

2.1. Concepto y sintaxis

Una **lista** es una estructura de datos **mutable** y **ordenada** que permite almacenar una colección de elementos. Es la estructura de datos más versátil y utilizada en Python.

Características principales:

- **Mutable**: Se puede modificar después de su creación (agregar, eliminar, cambiar elementos)
- **Ordenada**: Los elementos mantienen el orden de inserción
- **Indexada**: Cada elemento tiene una posición numérica (índice) que empieza en 0
- **Heterogénea**: Puede contener elementos de diferentes tipos (aunque no es recomendable)
- **Permite duplicados**: Un mismo valor puede aparecer múltiples veces

Sintaxis básica:

```
# Lista vacía
lista_vacia = []
otra_lista_vacia = list()

# Lista con elementos
numeros = [1, 2, 3, 4, 5]
nombres = ["Ana", "Carlos", "María"]
mixta = [42, "texto", 3.14, True]  # No recomendable, pero válido

# Lista creada con list() desde otras secuencias
letras = list("Python")  # ['P', 'y', 't', 'h', 'o', 'n']
rango = list(range(5))   # [0, 1, 2, 3, 4]
```

2.2. Creación y acceso a elementos

Creación de listas:

```
# Listas literales
frutas = ["manzana", "banana", "naranja"]
numeros = [10, 20, 30, 40, 50]

# Listas con repetición
ceros = [0] * 5          # [0, 0, 0, 0, 0]
patron = [1, 2] * 3      # [1, 2, 1, 2, 1, 2]

# Listas desde rangos
numeros_pares = list(range(0, 10, 2))  # [0, 2, 4, 6, 8]

# Listas por comprensión (se verá en detalle más adelante)
cuadrados = [x**2 for x in range(5)]   # [0, 1, 4, 9, 16]
```

Acceso a elementos mediante índices:

```
colores = ["rojo", "verde", "azul", "amarillo", "negro"]

# Acceso por índice positivo (desde el inicio)
primer_color = colores[0]   # "rojo"
segundo_color = colores[1]  # "verde"

# Intentar acceder a un índice fuera de rango produce error
# elemento = colores[10]    # IndexError: list index out of range

# Verificar longitud antes de acceder
if len(colores) > 2:
    tercer_color = colores[2]  # Seguro
```

Modificación de elementos:

```
# Las listas son mutables, podemos cambiar sus elementos
numeros = [1, 2, 3, 4, 5]
numeros[0] = 10        # Cambia el primer elemento
print(numeros)         # [10, 2, 3, 4, 5]

numeros[2] = 99        # Cambia el tercer elemento
print(numeros)         # [10, 2, 99, 4, 5]
```

2.3. Índices positivos y negativos

Python permite acceder a los elementos de una lista usando índices positivos (desde el inicio) o negativos (desde el final).

```
letras = ["A", "B", "C", "D", "E"]

# Índices positivos (empiezan en 0)
#  Índice:    0    1    2    3    4
#  Valor:    "A"  "B"  "C"  "D"  "E"

print(letras[0])   # "A" - primer elemento
print(letras[4])   # "E" - quinto elemento

# Índices negativos (empiezan en -1 desde el final)
#  Índice:   -5   -4   -3   -2   -1
#  Valor:    "A"  "B"  "C"  "D"  "E"

print(letras[-1])  # "E" - último elemento
print(letras[-2])  # "D" - penúltimo elemento
print(letras[-5])  # "A" - primer elemento (usando índice negativo)
```

Tabla comparativa de índices:

Expresión	Índice positivo	Índice negativo	Resultado
`letras[0]`	0	-5	"A"
`letras[1]`	1	-4	"B"
`letras[2]`	2	-3	"C"
`letras[3]`	3	-2	"D"

Expresión	Índice positivo	Índice negativo	Resultado
`letras[4]`	4	-1	"E"

Uso práctico de índices negativos:

```
numeros = [10, 20, 30, 40, 50]

# Acceder al último elemento sin saber la longitud
ultimo = numeros[-1]              # 50 (más legible que numeros[len(numeros)-1])

# Acceder a los últimos elementos
penultimo = numeros[-2]           # 40
antepenultimo = numeros[-3]       # 30

# Modificar desde el final
numeros[-1] = 99                  # Cambia el último: [10, 20, 30, 40, 99]
```

2.4. Métodos principales de listas

Las listas en Python tienen métodos integrados para manipular sus elementos. Estos métodos modifican la lista **in-place** (en su lugar) sin crear una nueva lista.

Métodos para agregar elementos

`append(elemento)` - Agrega un elemento al final de la lista

```
frutas = ["manzana", "banana"]
frutas.append("naranja")
print(frutas)  # ["manzana", "banana", "naranja"]

# append() agrega el elemento completo, incluso si es una lista
frutas.append(["uva", "pera"])
print(frutas)  # ["manzana", "banana", "naranja", ["uva", "pera"]]
```

`extend(iterable)` - Extiende la lista agregando elementos de un iterable

```python
numeros = [1, 2, 3]
numeros.extend([4, 5, 6])
print(numeros)  # [1, 2, 3, 4, 5, 6]

# Diferencia entre append() y extend()
lista_a = [1, 2, 3]
lista_a.append([4, 5])
print(lista_a)  # [1, 2, 3, [4, 5]] - agrega la lista como un elemento

lista_b = [1, 2, 3]
lista_b.extend([4, 5])
print(lista_b)  # [1, 2, 3, 4, 5] - agrega los elementos individualmente
```

`insert(índice, elemento)` - Inserta un elemento en una posición específica

```python
colores = ["rojo", "verde", "azul"]
colores.insert(1, "amarillo")  # Inserta en posición 1
print(colores)  # ["rojo", "amarillo", "verde", "azul"]

# Si el índice es mayor que la longitud, inserta al final
colores.insert(100, "negro")
print(colores)  # ["rojo", "amarillo", "verde", "azul", "negro"]
```

Métodos para eliminar elementos

`remove(valor)` - Elimina la primera ocurrencia del valor especificado

```python
numeros = [1, 2, 3, 2, 4]
numeros.remove(2)  # Elimina el primer 2
print(numeros)     # [1, 3, 2, 4]

# Si el valor no existe, produce ValueError
# numeros.remove(99)  # ValueError: list.remove(x): x not in list

# Verificar antes de eliminar
if 3 in numeros:
    numeros.remove(3)
```

`pop(índice=-1)` - Elimina y devuelve el elemento en el índice especificado (por defecto el último)

```python
frutas = ["manzana", "banana", "naranja", "uva"]

# pop() sin argumentos elimina y devuelve el último elemento
ultima_fruta = frutas.pop()
print(ultima_fruta)  # "uva"
print(frutas)        # ["manzana", "banana", "naranja"]

# pop(índice) elimina y devuelve el elemento en esa posición
primera_fruta = frutas.pop(0)
print(primera_fruta)  # "manzana"
print(frutas)         # ["banana", "naranja"]
```

`clear()` - Elimina todos los elementos de la lista

```python
numeros = [1, 2, 3, 4, 5]
numeros.clear()
print(numeros)  # []
```

Métodos de ordenamiento y reorganización

`sort(key=None, reverse=False)` - Ordena la lista in-place

```python
# Ordenamiento ascendente (por defecto)
numeros = [3, 1, 4, 1, 5, 9, 2]
numeros.sort()
print(numeros)  # [1, 1, 2, 3, 4, 5, 9]

# Ordenamiento descendente
numeros.sort(reverse=True)
print(numeros)  # [9, 5, 4, 3, 2, 1, 1]

# Ordenamiento de strings (alfabético)
palabras = ["zebra", "ana", "Python", "manzana"]
palabras.sort()
print(palabras)  # ["Python", "ana", "manzana", "zebra"] - mayúsculas primero

# Ordenamiento ignorando mayúsculas/minúsculas
palabras.sort(key=str.lower)
print(palabras)  # ["ana", "manzana", "Python", "zebra"]

# Ordenamiento por longitud
palabras.sort(key=len)
print(palabras)  # ["ana", "zebra", "Python", "manzana"]
```

`reverse()` - Invierte el orden de los elementos in-place

```python
numeros = [1, 2, 3, 4, 5]
numeros.reverse()
print(numeros)  # [5, 4, 3, 2, 1]

# No confundir con sorted() y reversed() que crean nuevas listas
```

`copy()` - Crea una copia superficial de la lista

```python
original = [1, 2, 3]
copia = original.copy()

copia.append(4)
print(original)  # [1, 2, 3] - no se modifica
print(copia)     # [1, 2, 3, 4]

# Equivalente a:
copia2 = original[:]
copia3 = list(original)
```

Métodos de búsqueda y conteo

`count(valor)` - Cuenta cuántas veces aparece un valor

```python
numeros = [1, 2, 2, 3, 2, 4, 2]
cantidad_doses = numeros.count(2)
print(cantidad_doses)  # 4

cantidad_cincos = numeros.count(5)
print(cantidad_cincos)  # 0 - devuelve 0 si no existe
```

`index(valor, inicio=0, fin=len)` - Devuelve el índice de la primera ocurrencia

```python
letras = ["a", "b", "c", "b", "d"]

# Buscar primera ocurrencia
posicion = letras.index("b")
print(posicion)  # 1

# Buscar desde una posición específica
posicion_siguiente = letras.index("b", 2)  # Busca desde índice 2
print(posicion_siguiente)  # 3

# Si no existe, produce ValueError
# posicion = letras.index("z")  # ValueError: 'z' is not in list

# Verificar antes de buscar
if "c" in letras:
    posicion_c = letras.index("c")
```

Tabla resumen de métodos:

Método	Modifica lista	Devuelve valor	Descripción
`append(x)`	Sí	None	Agrega elemento al final
`extend(iter)`	Sí	None	Agrega elementos de un iterable
`insert(i, x)`	Sí	None	Inserta elemento en posición i
`remove(x)`	Sí	None	Elimina primera ocurrencia de x
`pop(i)`	Sí	Elemento	Elimina y devuelve elemento en posición i
`clear()`	Sí	None	Elimina todos los elementos
`sort()`	Sí	None	Ordena la lista in-place
`reverse()`	Sí	None	Invierte el orden
`copy()`	No	Nueva lista	Crea una copia superficial
`count(x)`	No	int	Cuenta ocurrencias de x
`index(x)`	No	int	Devuelve índice de x

2.5. Operaciones con listas

Además de los métodos, las listas soportan operadores y funciones integradas.

Concatenación (+)

```
lista1 = [1, 2, 3]
lista2 = [4, 5, 6]
resultado = lista1 + lista2
print(resultado)  # [1, 2, 3, 4, 5, 6]

# Las listas originales no se modifican
print(lista1)  # [1, 2, 3]

# Concatenar múltiples listas
todas = [1, 2] + [3, 4] + [5, 6]
print(todas)  # [1, 2, 3, 4, 5, 6]
```

Repetición (`*`)

```python
patron = [0, 1]
repetido = patron * 4
print(repetido)  # [0, 1, 0, 1, 0, 1, 0, 1]

# Crear lista con valores iniciales
ceros = [0] * 5
print(ceros)  # [0, 0, 0, 0, 0]

# Cuidado con objetos mutables
lista_listas = [[]] * 3  # ¡PELIGRO! Las tres sublistas son el mismo objeto
lista_listas[0].append(1)
print(lista_listas)  # [[1], [1], [1]] - todas se modificaron

# Forma correcta:
lista_listas_correcta = [[] for _ in range(3)]
lista_listas_correcta[0].append(1)
print(lista_listas_correcta)  # [[1], [], []] - solo se modificó la primera
```

Pertenencia (`in` , `not in`)

```python
frutas = ["manzana", "banana", "naranja"]

# Verificar si un elemento existe
if "banana" in frutas:
    print("Tenemos bananas")  # Se ejecuta

if "uva" not in frutas:
    print("No tenemos uvas")  # Se ejecuta

# Útil para validaciones
def agregar_si_no_existe(lista: list, elemento) -> None:
    if elemento not in lista:
        lista.append(elemento)

numeros = [1, 2, 3]
agregar_si_no_existe(numeros, 2)  # No se agrega (ya existe)
agregar_si_no_existe(numeros, 4)  # Se agrega
print(numeros)  # [1, 2, 3, 4]
```

Longitud (`len()`)

```python
colores = ["rojo", "verde", "azul"]
cantidad = len(colores)
print(cantidad)  # 3

# Útil para iterar con índices
for i in range(len(colores)):
    print(f"Color {i}: {colores[i]}")

# Verificar si lista está vacía
lista_vacia = []
if len(lista_vacia) == 0:
    print("La lista está vacía")

# Forma más pythónica:
if not lista_vacia:
    print("La lista está vacía")
```

Funciones integradas útiles

```
numeros = [3, 1, 4, 1, 5, 9, 2, 6]

# Mínimo y máximo
minimo = min(numeros)   # 1
maximo = max(numeros)   # 9

# Suma de elementos
total = sum(numeros)    # 31

# Ordenamiento sin modificar la original (devuelve nueva lista)
ordenada = sorted(numeros)
print(ordenada)         # [1, 1, 2, 3, 4, 5, 6, 9]
print(numeros)          # [3, 1, 4, 1, 5, 9, 2, 6] - sin cambios

# Invertir sin modificar la original
invertida = list(reversed(numeros))
print(invertida)        # [6, 2, 9, 5, 1, 4, 1, 3]

# any() y all()
booleanos = [True, True, False, True]
print(any(booleanos))   # True - al menos uno es True
print(all(booleanos))   # False - no todos son True

# Verificar si todos los números son positivos
numeros_positivos = [1, 2, 3, 4]
todos_positivos = all(n > 0 for n in numeros_positivos)
print(todos_positivos)  # True
```

2.6. Listas anidadas

Una lista puede contener otras listas como elementos, creando estructuras multidimensionales (matrices, tablas, etc.).

```python
# Matriz 3x3 (lista de listas)
matriz = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Acceso a elementos
primer_fila = matriz[0]        # [1, 2, 3]
elemento_central = matriz[1][1] # 5
esquina_inferior_derecha = matriz[2][2]  # 9

# Modificar elemento
matriz[0][0] = 99
print(matriz[0])  # [99, 2, 3]

# Iterar sobre matriz
for fila in matriz:
    for elemento in fila:
        print(elemento, end=" ")
    print()  # Nueva línea después de cada fila

# Salida:
# 99 2 3
# 4 5 6
# 7 8 9
```

Ejemplo práctico: Lista de estudiantes

```python
# Cada estudiante es una lista [nombre, edad, nota]
estudiantes = [
    ["Ana", 20, 8.5],
    ["Carlos", 22, 7.0],
    ["María", 21, 9.2],
    ["Juan", 23, 6.8]
]

# Acceder a información específica
nombre_primero = estudiantes[0][0]  # "Ana"
nota_tercero = estudiantes[2][2]    # 9.2

# Calcular promedio de notas
suma_notas = sum(estudiante[2] for estudiante in estudiantes)
promedio = suma_notas / len(estudiantes)
print(f"Promedio de notas: {promedio:.2f}")  # 7.88

# Buscar estudiantes con nota >= 8
aprobados_excelentes = [est[0] for est in estudiantes if est[2] >= 8]
print(aprobados_excelentes)  # ["Ana", "María"]
```

Creación de matrices:

```python
# Forma incorrecta (crea referencias al mismo objeto)
matriz_mala = [[0] * 3] * 3
matriz_mala[0][0] = 1
print(matriz_mala)  # [[1, 0, 0], [1, 0, 0], [1, 0, 0]] - ¡todas las filas cambiaron!

# Forma correcta (crea objetos independientes)
matriz_buena = [[0] * 3 for _ in range(3)]
matriz_buena[0][0] = 1
print(matriz_buena)  # [[1, 0, 0], [0, 0, 0], [0, 0, 0]] - solo cambió la primera fila
```

2.7. Ejercicios prácticos

Ejercicio 2.1: Gestión de lista de tareas

Crea un programa que permita gestionar una lista de tareas con las siguientes opciones:

- Agregar tarea
- Eliminar tarea

- Mostrar todas las tareas
- Marcar tarea como completada (moverla a otra lista)

```python
# Solución sugerida
tareas_pendientes = []
tareas_completadas = []

def agregar_tarea(tarea: str) -> None:
    tareas_pendientes.append(tarea)
    print(f"Tarea '{tarea}' agregada")

def eliminar_tarea(tarea: str) -> None:
    if tarea in tareas_pendientes:
        tareas_pendientes.remove(tarea)
        print(f"Tarea '{tarea}' eliminada")
    else:
        print("Tarea no encontrada")

def mostrar_tareas() -> None:
    print("\nTareas pendientes:")
    for i, tarea in enumerate(tareas_pendientes, 1):
        print(f"{i}. {tarea}")

    print("\nTareas completadas:")
    for i, tarea in enumerate(tareas_completadas, 1):
        print(f"{i}. {tarea}")

def completar_tarea(tarea: str) -> None:
    if tarea in tareas_pendientes:
        tareas_pendientes.remove(tarea)
        tareas_completadas.append(tarea)
        print(f"Tarea '{tarea}' marcada como completada")
    else:
        print("Tarea no encontrada")
```

Ejercicio 2.2: Estadísticas de calificaciones

Dada una lista de calificaciones, calcula:

- Promedio
- Nota más alta y más baja
- Cantidad de aprobados (>= 5) y suspensos

- Calificaciones ordenadas

```python
calificaciones = [7.5, 4.2, 8.9, 6.0, 3.5, 9.1, 5.8, 7.0]

# Calcular estadísticas
promedio = sum(calificaciones) / len(calificaciones)
nota_maxima = max(calificaciones)
nota_minima = min(calificaciones)
aprobados = sum(1 for nota in calificaciones if nota >= 5)
suspensos = len(calificaciones) - aprobados
ordenadas = sorted(calificaciones, reverse=True)

print(f"Promedio: {promedio:.2f}")
print(f"Nota máxima: {nota_maxima}")
print(f"Nota mínima: {nota_minima}")
print(f"Aprobados: {aprobados}, Suspensos: {suspensos}")
print(f"Calificaciones ordenadas: {ordenadas}")
```

Ejercicio 2.3: Eliminar duplicados

Escribe una función que elimine los elementos duplicados de una lista manteniendo el orden original.

```python
def eliminar_duplicados(lista: list) -> list:
    resultado = []
    for elemento in lista:
        if elemento not in resultado:
            resultado.append(elemento)
    return resultado

# Prueba
numeros = [1, 2, 2, 3, 4, 3, 5, 1, 6]
sin_duplicados = eliminar_duplicados(numeros)
print(sin_duplicados)  # [1, 2, 3, 4, 5, 6]
```

Ejercicio 2.4: Rotación de lista

Crea una función que rote una lista n posiciones a la derecha.

```python
def rotar_lista(lista: list, n: int) -> list:
    """Rota la lista n posiciones a la derecha"""
    if not lista or n == 0:
        return lista

    # Normalizar n (si n > len(lista), usar el módulo)
    n = n % len(lista)

    # Rotar: tomar últimos n elementos y ponerlos al inicio
    return lista[-n:] + lista[:-n]

# Prueba
numeros = [1, 2, 3, 4, 5]
rotada = rotar_lista(numeros, 2)
print(rotada)  # [4, 5, 1, 2, 3]
```

Ejercicio 2.5: Matriz transpuesta

Dada una matriz (lista de listas), calcula su transpuesta.

```python
def transponer_matriz(matriz: list[list]) -> list[list]:
    """Calcula la transpuesta de una matriz"""
    filas = len(matriz)
    columnas = len(matriz[0])

    # Crear matriz transpuesta
    transpuesta = [[matriz[j][i] for j in range(filas)]
                   for i in range(columnas)]

    return transpuesta

# Prueba
matriz = [
    [1, 2, 3],
    [4, 5, 6]
]

resultado = transponer_matriz(matriz)
print("Original:")
for fila in matriz:
    print(fila)

print("\nTranspuesta:")
for fila in resultado:
    print(fila)

# Salida:
# Original:
# [1, 2, 3]
# [4, 5, 6]
#
# Transpuesta:
# [1, 4]
# [2, 5]
# [3, 6]
```

3. Bucle while

3.1. Concepto y sintaxis

El bucle `while` es una estructura de control iterativa que repite un bloque de código **mientras** una condición sea verdadera. A diferencia del bucle `for` (que itera sobre una secuencia conocida), el `while` continúa ejecutándose hasta que la condición evaluada sea `False`.

Sintaxis básica:

```
while condición:
    # Bloque de código que se repite
    # mientras la condición sea True
```

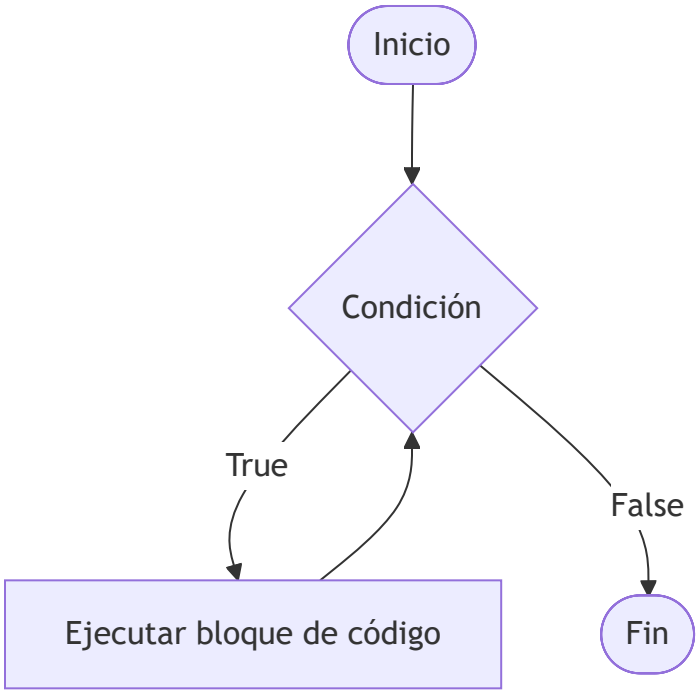
Características principales:

- Evalúa la condición **antes** de cada iteración
- Si la condición es `False` desde el inicio, el bloque nunca se ejecuta
- Es responsabilidad del programador modificar la condición dentro del bucle para evitar bucles infinitos
- Útil cuando no se conoce de antemano cuántas iteraciones son necesarias

Ejemplo básico:

```
# Contar del 1 al 5
contador = 1
while contador <= 5:
    print(contador)
    contador += 1  # IMPORTANTE: modificar la variable de control

# Salida:
# 1
# 2
# 3
# 4
# 5
```

Diagrama de flujo del while:



3.2. Condiciones de continuación y terminación

La condición del `while` puede ser cualquier expresión que evalúe a un booleano. El bucle continúa mientras la condición sea `True` y termina cuando sea `False`.

Tipos de condiciones comunes:

```python
# 1. Condición numérica
contador = 0
while contador < 10:
    print(contador)
    contador += 1

# 2. Condición booleana
ejecutar = True
while ejecutar:
    respuesta = input("¿Continuar? (s/n): ")
    ejecutar = respuesta.lower() == "s"

# 3. Condición de lista no vacía
numeros = [1, 2, 3, 4, 5]
while numeros:  # Mientras la lista no esté vacía
    print(numeros.pop())  # Elimina y muestra el último

# 4. Condición compuesta
intentos = 0
max_intentos = 3
exito = False
while intentos < max_intentos and not exito:
    respuesta = input("Adivina el número (1-10): ")
    if respuesta == "7":
        exito = True
        print("¡Correcto!")
    else:
        intentos += 1
        print(f"Intento {intentos}/{max_intentos}")
```

Condiciones de terminación:

Un bucle `while` termina cuando:

1. La condición se evalúa como `False`
2. Se ejecuta `break` (salida forzada)
3. Se produce una excepción no capturada
4. El programa termina

```python
# Ejemplo: Validación de entrada
edad = -1
while edad < 0 or edad > 120:  # Condición de continuación
    edad = int(input("Introduce tu edad (0-120): "))
    if edad < 0 or edad > 120:
        print("Edad inválida. Intenta de nuevo.")

print(f"Edad válida: {edad}")
```

3.3. Bucles infinitos y cómo evitarlos

Un **bucle infinito** ocurre cuando la condición del `while` nunca se vuelve `False`, causando que el programa se ejecute indefinidamente.

Bucles infinitos intencionales:

En algunos casos, los bucles infinitos son deseables (por ejemplo, en servidores):

```python
# Servidor web simplificado
while True:
    solicitud = esperar_solicitud()
    procesar(solicitud)
    enviar_respuesta()

# Menú interactivo
while True:
    print("\n--- MENÚ ---")
    print("1. Opción A")
    print("2. Opción B")
    print("3. Salir")

    opcion = input("Selecciona: ")

    if opcion == "1":
        print("Ejecutando opción A")
    elif opcion == "2":
        print("Ejecutando opción B")
    elif opcion == "3":
        print("Saliendo...")
        break
    else:
        print("Opción inválida")
```

3.4. while con else

Python permite agregar una cláusula `else` al bucle `while`. El bloque `else` se ejecuta **solo si el bucle termina normalmente** (sin `break`).

Sintaxis:

```python
while condición:
    # Bloque que se repite
else:
    # Se ejecuta si el while termina sin break
```

Ejemplo comparativo:

```python
# Buscar un número en una lista
numeros = [1, 3, 5, 7, 9]
objetivo = 5
indice = 0

while indice < len(numeros):
    if numeros[indice] == objetivo:
        print(f"Encontrado en posición {indice}")
        break
    indice += 1
else:
    # Se ejecuta solo si NO se encontró (no hubo break)
    print(f"No se encontró el número {objetivo}")

# Ejemplo 2: Validación con límite de intentos
intentos = 0
max_intentos = 3
contraseña_correcta = "python123"

while intentos < max_intentos:
    contraseña = input("Introduce la contraseña: ")
    if contraseña == contraseña_correcta:
        print("Acceso concedido")
        break
    intentos += 1
    print(f"Contraseña incorrecta. Intentos restantes: {max_intentos - intentos}")
else:
    # Se ejecuta solo si se agotaron los intentos (sin break)
    print("Acceso denegado. Demasiados intentos fallidos.")
```

Cuándo usar `while-else`:

- Búsqueda de elementos (para detectar si no se encontró)
- Validación con límite de intentos
- Algoritmos que necesitan distinguir entre "terminó normalmente" vs "salió anticipadamente"

Comparación con y sin else:

```python
# Sin else (menos elegante)
encontrado = False
i = 0
while i < len(lista):
    if lista[i] == objetivo:
        encontrado = True
        break
    i += 1

if not encontrado:
    print("No encontrado")

# Con else (más pythónico)
i = 0
while i < len(lista):
    if lista[i] == objetivo:
        break
    i += 1
else:
    print("No encontrado")
```

3.5. Comparación while vs. for

Tanto `while` como `for` permiten repetir código, pero tienen usos diferentes según el contexto.

Tabla comparativa:

Característica	`while`	`for`
Uso principal	Condición desconocida	Iterar sobre secuencia
Iteraciones	Desconocidas de antemano	Conocidas (longitud de secuencia)
Control	Manual (modificar variable)	Automático (siguiente elemento)
Legibilidad	Puede ser menos clara	Generalmente más clara
Riesgo bucle infinito	Alto (si no se actualiza condición)	Bajo (termina con la secuencia)

Característica	`while`	`for`
Mejor para	Esperas, validaciones, condiciones complejas	Recorrer listas, rangos, archivos

Ejemplo 1: Contar del 1 al 10

```
# Con while (manual)
contador = 1
while contador <= 10:
    print(contador)
    contador += 1

# Con for (automático, más legible)
for contador in range(1, 11):
    print(contador)
```

Ejemplo 2: Recorrer una lista

```
frutas = ["manzana", "banana", "naranja"]

# Con while (menos pythónico)
i = 0
while i < len(frutas):
    print(frutas[i])
    i += 1

# Con for (más pythónico)
for fruta in frutas:
    print(fruta)
```

Cuándo usar `while`:

1. **Esperar una condición**: Entrada del usuario, evento externo

```
respuesta = ""
while respuesta != "salir":
    respuesta = input("Comando: ")
```

2. **Validación de entrada**: No sabes cuántos intentos necesitarás

```python
numero = -1
while numero < 1 or numero > 100:
    numero = int(input("Número entre 1 y 100: "))
```

3. **Algoritmos con condición compleja**: Búsqueda, convergencia

```python
diferencia = float('inf')
while diferencia > 0.001:  # Hasta que converja
    nuevo_valor = calcular()
    diferencia = abs(nuevo_valor - valor_anterior)
    valor_anterior = nuevo_valor
```

4. **Procesar hasta vaciar**: Consumir una cola, pila

```python
tareas = ["tarea1", "tarea2", "tarea3"]
while tareas:
    tarea_actual = tareas.pop(0)
    procesar(tarea_actual)
```

Cuándo usar `for`:

1. **Iterar sobre secuencia conocida**: Listas, tuplas, strings

```python
for elemento in lista:
    procesar(elemento)
```

2. **Número fijo de iteraciones**: Rangos numéricos

```python
for i in range(100):
    hacer_algo()
```

3. **Recorrer archivos**: Líneas de un archivo

```python
with open("archivo.txt") as f:
    for linea in f:
        print(linea)
```

Conversión entre while y for:

```python
# Muchos for pueden escribirse como while (pero no debería)
for i in range(5):
    print(i)

# Equivalente en while (menos legible)
i = 0
while i < 5:
    print(i)
    i += 1

# Los while con condiciones complejas NO pueden escribirse fácilmente como for
intentos = 0
exito = False
while intentos < 3 and not exito:
    # Este while NO tiene equivalente directo en for
    exito = intentar_operacion()
    intentos += 1
```

Regla práctica: Si sabes de antemano cuántas iteraciones necesitas o sobre qué vas a iterar, usa `for`. Si dependes de una condición que puede cambiar durante la ejecución, usa `while`.

3.6. Ejercicios prácticos

Ejercicio 3.1: Suma acumulativa

Pide números al usuario y suma todos hasta que escriba 0. Muestra el total al final.

```python
total = 0
numero = None  # Valor inicial diferente de 0

while numero != 0:
    numero = int(input("Introduce un número (0 para terminar): "))
    total += numero

print(f"La suma total es: {total}")
```

Ejercicio 3.2: Adivina el número

El programa elige un número aleatorio entre 1 y 100. El usuario debe adivinarlo con pistas de "mayor" o "menor".

```python
import random

numero_secreto = random.randint(1, 100)
intentos = 0
adivinado = False

print("He pensado un número entre 1 y 100. ¡Adivínalo!")

while not adivinado:
    intento = int(input("Tu intento: "))
    intentos += 1

    if intento < numero_secreto:
        print("El número es MAYOR")
    elif intento > numero_secreto:
        print("El número es MENOR")
    else:
        adivinado = True
        print(f"¡Correcto! Lo adivinaste en {intentos} intentos")
```

Ejercicio 3.3: Validación de contraseña robusta

Pide una contraseña que cumpla requisitos específicos: mínimo 8 caracteres, al menos una mayúscula, una minúscula y un número.

```python
def validar_contraseña(contraseña: str) -> tuple[bool, list[str]]:
    """Valida una contraseña y devuelve (es_válida, errores)"""
    errores = []

    if len(contraseña) < 8:
        errores.append("Debe tener al menos 8 caracteres")

    if not any(c.isupper() for c in contraseña):
        errores.append("Debe tener al menos una mayúscula")

    if not any(c.islower() for c in contraseña):
        errores.append("Debe tener al menos una minúscula")

    if not any(c.isdigit() for c in contraseña):
        errores.append("Debe tener al menos un número")

    return len(errores) == 0, errores

# Programa principal
contraseña_valida = False

while not contraseña_valida:
    contraseña = input("Crea una contraseña: ")
    valida, errores = validar_contraseña(contraseña)

    if valida:
        contraseña_valida = True
        print("¡Contraseña creada exitosamente!")
    else:
        print("Contraseña inválida:")
        for error in errores:
            print(f"  - {error}")
```

Ejercicio 3.4: Calculadora interactiva

Crea una calculadora que funcione hasta que el usuario escriba "salir".

```python
print("Calculadora interactiva")
print("Operaciones: +, -, *, /")
print("Escribe 'salir' para terminar\n")

continuar = True

while continuar:
    entrada = input("Operación (ej: 5 + 3) o 'salir': ")

    if entrada.lower() == "salir":
        continuar = False
        print("¡Hasta luego!")
    else:
        try:
            # Evaluar la expresión (¡ADVERTENCIA: eval() puede ser inseguro!)
            resultado = eval(entrada)
            print(f"Resultado: {resultado}\n")
        except Exception as e:
            print(f"Error: {e}\n")

# Versión más segura sin eval()
while continuar:
    entrada = input("Número 1: ")
    if entrada.lower() == "salir":
        break

    num1 = float(entrada)
    operacion = input("Operación (+, -, *, /): ")
    num2 = float(input("Número 2: "))

    if operacion == "+":
        print(f"Resultado: {num1 + num2}\n")
    elif operacion == "-":
        print(f"Resultado: {num1 - num2}\n")
    elif operacion == "*":
        print(f"Resultado: {num1 * num2}\n")
    elif operacion == "/":
        if num2 != 0:
            print(f"Resultado: {num1 / num2}\n")
        else:
            print("Error: División por cero\n")
```

```
    else:
        print("Operación inválida\n")
```

Ejercicio 3.5: Cuenta regresiva con pausa

Implementa una cuenta regresiva que permita pausar/reanudar con input.

```
import time

segundos = int(input("Segundos para la cuenta regresiva: "))

print("Presiona Enter para pausar/reanudar")
print("Iniciando cuenta regresiva...\n")

pausado = False

while segundos > 0:
    if not pausado:
        print(f"{segundos} segundos restantes", end="\r")
        time.sleep(1)
        segundos -= 1

    # Verificar si el usuario quiere pausar (sin bloquear)
    # Nota: Esta versión simplificada no permite pausa en tiempo real
    # Para pausa real se necesitaría threading o input no bloqueante

print("\n¡Tiempo terminado!")

# Versión con pausa manual (espera input del usuario)
segundos = int(input("Segundos: "))

while segundos > 0:
    print(f"{segundos} segundos restantes")

    accion = input("Presiona Enter para continuar (o 'salir'): ")
    if accion.lower() == "salir":
        print("Cuenta regresiva cancelada")
        break

    segundos -= 1
else:
    print("¡Tiempo terminado!")
```

Ejercicio 3.6: Simulación de cola de atención

Simula una cola de clientes donde se atienden uno por uno hasta que la cola esté vacía.

```
cola_clientes = ["Cliente A", "Cliente B", "Cliente C", "Cliente D", "Cliente E"]
atendidos = 0

print("=== SISTEMA DE ATENCIÓN AL CLIENTE ===\n")
print(f"Clientes en espera: {len(cola_clientes)}\n")

while cola_clientes:
    print(f"Clientes en cola: {', '.join(cola_clientes)}")

    # Atender al primer cliente
    cliente_actual = cola_clientes.pop(0)
    print(f"\nAtendiendo a: {cliente_actual}")

    # Simular tiempo de atención
    input("Presiona Enter cuando termine la atención...")

    atendidos += 1
    print(f"✓ {cliente_actual} atendido\n")
else:
    print(f"=== ATENCIÓN COMPLETADA ===")
    print(f"Total de clientes atendidos: {atendidos}")
```

4. Bucle for

4.1. Concepto y sintaxis

El bucle `for` es una estructura de control iterativa que recorre secuencialmente los elementos de un objeto iterable (listas, tuplas, strings, rangos, etc.). Es la forma más natural y pythónica de iterar en Python.

Sintaxis básica:

```python
for variable in secuencia:
    # Bloque de código que se ejecuta
    # para cada elemento de la secuencia
```

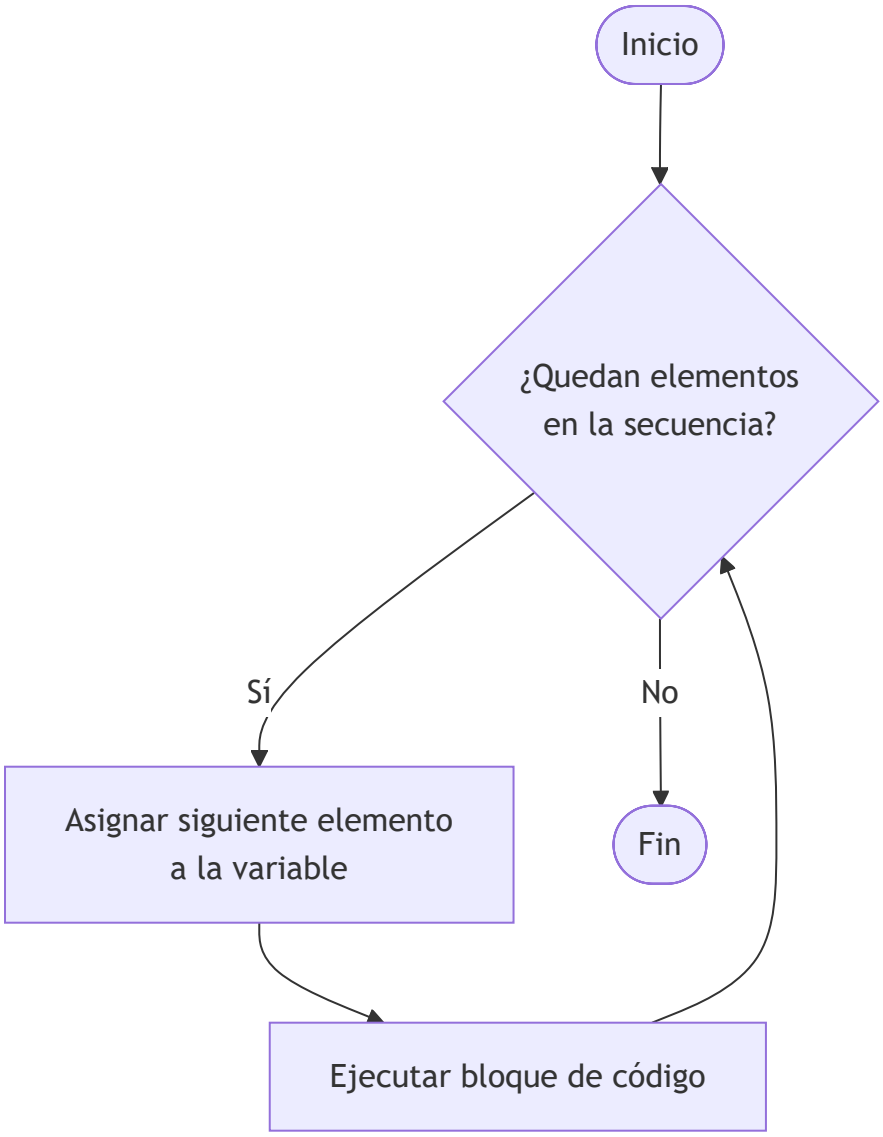
Características principales:

- Itera **automáticamente** sobre cada elemento de una secuencia
- La variable de iteración toma el valor de cada elemento sucesivamente
- No requiere índices ni contadores manuales (aunque se pueden usar)
- Más legible y menos propenso a errores que `while` para iteraciones conocidas
- Termina automáticamente cuando se recorren todos los elementos

Ejemplos básicos:

```python
# Iterar sobre una lista
frutas = ["manzana", "banana", "naranja"]
for fruta in frutas:
    print(fruta)

# Salida:
# manzana
# banana
# naranja

# Iterar sobre un string (cada carácter)
palabra = "Python"
for letra in palabra:
    print(letra)

# Salida:
# P
# y
# t
# h
# o
# n

# Iterar sobre un rango numérico
for numero in range(5):
    print(numero)

# Salida:
# 0
# 1
# 2
# 3
# 4
```

Diagrama de flujo del for:



4.2. Iteración sobre listas

La iteración sobre listas es el uso más común del bucle `for`. Python permite recorrer directamente los elementos o usar índices cuando sea necesario.

Iteración directa sobre elementos:

```python
# Forma pythónica (recomendada)
numeros = [10, 20, 30, 40, 50]
for numero in numeros:
    print(numero)

# Operar con cada elemento
precios = [10.5, 20.0, 15.75, 30.25]
for precio in precios:
    precio_con_iva = precio * 1.21
    print(f"Precio: {precio}€ → Con IVA: {precio_con_iva:.2f}€")

# Modificar elementos (¡CUIDADO! No modifica la lista original)
numeros = [1, 2, 3, 4, 5]
for num in numeros:
    num = num * 2  # Esto NO modifica la lista
print(numeros)  # [1, 2, 3, 4, 5] - sin cambios
```

Iteración con índices (cuando se necesitan):

```python
# Usando range(len())
nombres = ["Ana", "Carlos", "María"]
for i in range(len(nombres)):
    print(f"Posición {i}: {nombres[i]}")

# Salida:
# Posición 0: Ana
# Posición 1: Carlos
# Posición 2: María

# Modificar lista in-place con índices
numeros = [1, 2, 3, 4, 5]
for i in range(len(numeros)):
    numeros[i] = numeros[i] * 2
print(numeros)  # [2, 4, 6, 8, 10] - modificada
```

Iteración con enumerate() (mejor práctica):

```python
# enumerate() devuelve (índice, elemento)
frutas = ["manzana", "banana", "naranja"]
for indice, fruta in enumerate(frutas):
    print(f"{indice}: {fruta}")

# Salida:
# 0: manzana
# 1: banana
# 2: naranja

# Empezar el índice en 1
for numero, fruta in enumerate(frutas, start=1):
    print(f"{numero}. {fruta}")

# Salida:
# 1. manzana
# 2. banana
# 3. naranja
```

Iterar sobre múltiples listas simultáneamente con zip():

```python
# zip() combina múltiples iterables
nombres = ["Ana", "Carlos", "María"]
edades = [25, 30, 22]
ciudades = ["Madrid", "Barcelona", "Valencia"]

for nombre, edad, ciudad in zip(nombres, edades, ciudades):
    print(f"{nombre} tiene {edad} años y vive en {ciudad}")

# Salida:
# Ana tiene 25 años y vive en Madrid
# Carlos tiene 30 años y vive en Barcelona
# María tiene 22 años y vive en Valencia

# Si las listas tienen diferente longitud, zip() se detiene en la más corta
lista1 = [1, 2, 3, 4, 5]
lista2 = ['a', 'b', 'c']
for num, letra in zip(lista1, lista2):
    print(num, letra)

# Salida:
# 1 a
# 2 b
# 3 c
```

Filtrado durante la iteración:

```python
# Procesar solo elementos que cumplan una condición
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
for num in numeros:
    if num % 2 == 0:  # Solo pares
        print(num)

# Salida: 2, 4, 6, 8, 10

# Crear nueva lista filtrada
pares = []
for num in numeros:
    if num % 2 == 0:
        pares.append(num)
print(pares)  # [2, 4, 6, 8, 10]

# Forma más pythónica con list comprehension
pares = [num for num in numeros if num % 2 == 0]
```

4.3. La función range()

`range()` genera secuencias numéricas inmutables. Es fundamental para crear bucles `for` con números.

Sintaxis de range():

```python
range(stop)           # De 0 hasta stop-1
range(start, stop)    # De start hasta stop-1
range(start, stop, step)  # De start hasta stop-1, incrementando step
```

`range(stop)` - Desde 0 hasta n-1

```python
# range(5) genera: 0, 1, 2, 3, 4
for i in range(5):
    print(i)

# Salida:
# 0
# 1
# 2
# 3
# 4

# Repetir una acción n veces (sin usar la variable)
for _ in range(3):
    print("Hola")

# Salida:
# Hola
# Hola
# Hola
```

`range(start, stop)` - Desde start hasta stop-1

```python
# range(1, 6) genera: 1, 2, 3, 4, 5
for i in range(1, 6):
    print(i)

# Salida:
# 1
# 2
# 3
# 4
# 5

# Tabla de multiplicar
numero = 7
for i in range(1, 11):
    print(f"{numero} × {i} = {numero * i}")

# Salida:
# 7 × 1 = 7
# 7 × 2 = 14
# ...
# 7 × 10 = 70
```

`range(start, stop, step)` - Con incremento personalizado

```python
# Números pares del 0 al 10
for i in range(0, 11, 2):
    print(i)

# Salida: 0, 2, 4, 6, 8, 10

# Números impares del 1 al 10
for i in range(1, 11, 2):
    print(i)

# Salida: 1, 3, 5, 7, 9

# Contar de 5 en 5
for i in range(0, 51, 5):
    print(i)

# Salida: 0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50

# Cuenta regresiva (step negativo)
for i in range(10, 0, -1):
    print(i)
print("¡Despegue!")

# Salida:
# 10
# 9
# ...
# 1
# ¡Despegue!

# Recorrer lista al revés por índices
numeros = [10, 20, 30, 40, 50]
for i in range(len(numeros) - 1, -1, -1):
    print(numeros[i])

# Salida: 50, 40, 30, 20, 10
```

Características de range():

```python
# range() NO es una lista, es un objeto range (más eficiente)
r = range(5)
print(type(r))  # <class 'range'>
print(r)        # range(0, 5)

# Convertir a lista si es necesario
print(list(r))  # [0, 1, 2, 3, 4]

# Verificar pertenencia
if 3 in range(10):
    print("3 está en el rango")

# Longitud del rango
print(len(range(1, 11)))  # 10

# Indexar un rango
r = range(10, 20, 2)
print(r[0])   # 10
print(r[-1])  # 18
```

Casos de uso comunes:

```python
# 1. Repetir n veces sin usar el índice
for _ in range(10):
    print("*", end="")
# Salida: **********

# 2. Índices para modificar lista
numeros = [1, 2, 3, 4, 5]
for i in range(len(numeros)):
    numeros[i] = numeros[i] ** 2
print(numeros)  # [1, 4, 9, 16, 25]

# 3. Generar lista de números
cuadrados = [i**2 for i in range(1, 11)]
print(cuadrados)  # [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

# 4. Sumar números en un rango
suma = sum(range(1, 101))  # Suma de 1 a 100
print(suma)  # 5050
```

4.4. for con else

Al igual que `while`, el bucle `for` puede tener una cláusula `else` que se ejecuta cuando el bucle termina normalmente (sin `break`).

Sintaxis:

```
for elemento in secuencia:
    # Bloque que se repite
else:
    # Se ejecuta si el for termina sin break
```

Ejemplo: Búsqueda en lista

```
# Buscar un elemento
numeros = [1, 3, 5, 7, 9]
objetivo = 5

for num in numeros:
    if num == objetivo:
        print(f"Encontrado: {objetivo}")
        break
else:
    # Se ejecuta solo si NO se encontró (no hubo break)
    print(f"No se encontró {objetivo}")

# Salida: Encontrado: 5

# Si el objetivo no existe
objetivo = 4
for num in numeros:
    if num == objetivo:
        print(f"Encontrado: {objetivo}")
        break
else:
    print(f"No se encontró {objetivo}")

# Salida: No se encontró 4
```

Ejemplo: Validación de lista

```python
# Verificar que todos los números sean positivos
numeros = [1, 2, 3, 4, 5]

for num in numeros:
    if num <= 0:
        print("Error: Se encontró un número no positivo")
        break
else:
    print("Todos los números son positivos")

# Salida: Todos los números son positivos
```

Ejemplo: Números primos

```python
def es_primo(n: int) -> bool:
    """Verifica si un número es primo."""
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False  # No es primo, salimos
    else:
        return True  # Es primo (el bucle terminó sin break)

# Prueba
print(es_primo(17))  # True
print(es_primo(20))  # False
```

5. Conceptos avanzados de bucles

5.1. Control de flujo en bucles

Python proporciona tres sentencias para controlar el flujo dentro de bucles: `break`, `continue` y `pass`.

`break` - Salir del bucle

Termina inmediatamente el bucle más interno y continúa con la siguiente instrucción después del bucle.

```python
# Salir al encontrar un elemento
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
for num in numeros:
    if num == 5:
        print("Encontré el 5, saliendo")
        break
    print(num)

# Salida: 1, 2, 3, 4
# Encontré el 5, saliendo
```

`continue` - Saltar a la siguiente iteración

Salta el resto del código en la iteración actual y continúa con la siguiente iteración del bucle.

```python
# Saltar números pares
for i in range(10):
    if i % 2 == 0:
        continue  # Salta los pares
    print(i)

# Salida: 1, 3, 5, 7, 9
```

`pass` - No hacer nada (marcador de posición)

Es una operación nula que no hace nada. Se usa como marcador de posición.

```python
# Marcador de posición durante desarrollo
for i in range(5):
    if i == 3:
        pass  # TODO: Implementar esto después
    else:
        print(i)

# Salida: 0, 1, 2, 4
```

Tabla resumen:

Sentencia	Acción	Continúa con
`break`	Sale del bucle completamente	Código después del bucle

Sentencia	Acción	Continúa con
`continue`	Salta al inicio de la siguiente iteración	Siguiente iteración del bucle
`pass`	No hace nada	Siguiente línea normalmente

5.2. Bucles con else

Los bucles `while` y `for` pueden tener una cláusula `else` que se ejecuta cuando el bucle termina normalmente (sin `break`).

Sintaxis

```
while condición:
    # código
else:
    # se ejecuta si NO hubo break

for elemento in secuencia:
    # código
else:
    # se ejecuta si NO hubo break
```

Ejemplo: Búsqueda con while

```
numeros = [2, 4, 6, 8, 10]
objetivo = 7
indice = 0

while indice < len(numeros):
    if numeros[indice] == objetivo:
        print(f"Encontrado en posición {indice}")
        break
    indice += 1
else:
    print(f"No se encontró {objetivo}")

# Salida: No se encontró 7
```

Ejemplo: Búsqueda con for

```
nombres = ["Ana", "Carlos", "María", "Juan"]
buscar = "Pedro"

for nombre in nombres:
    if nombre == buscar:
        print(f"✓ Encontrado: {nombre}")
        break
else:
    print(f"✗ No encontrado: {buscar}")

# Salida: ✗ No encontrado: Pedro
```

5.3. Bucles anidados

Un bucle anidado es un bucle dentro de otro bucle. El bucle interior se ejecuta completamente por cada iteración del bucle exterior.

Bucles for anidados

```
# Tabla de multiplicar
for i in range(1, 4):  # Bucle exterior
    for j in range(1, 4):  # Bucle interior
        print(f"{i} × {j} = {i*j}")
    print()  # Línea en blanco entre tablas

# Salida:
# 1 × 1 = 1
# 1 × 2 = 2
# 1 × 3 = 3
#
# 2 × 1 = 2
# 2 × 2 = 4
# 2 × 3 = 6
# ...
```

Ejemplo: Matriz

```python
# Crear e imprimir una matriz
matriz = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

for fila in matriz:
    for elemento in fila:
        print(elemento, end=" ")
    print()  # Nueva línea después de cada fila

# Salida:
# 1 2 3
# 4 5 6
# 7 8 9
```

Ejemplo: Patrón de asteriscos

```python
# Triángulo de asteriscos
for i in range(1, 6):
    for j in range(i):
        print("*", end="")
    print()

# Salida:
# *
# **
# ***
# ****
# *****
```

6. Ejercicios prácticos

6.1. Ejercicios de listas

Ejercicio 1.1: Operaciones básicas

Crea una lista con los números del 1 al 10, luego:

- Agrega el número 11 al final
- Elimina el número 5
- Ordena la lista en orden descendente
- Muestra la longitud de la lista

Ejercicio 1.2: Filtrar elementos

Dada una lista de números, crea una nueva lista que contenga solo los números mayores que 50.

```
numeros = [23, 67, 45, 89, 12, 55, 78, 34, 91]
# Tu código aquí
```

Ejercicio 1.3: Promedio de calificaciones

Calcula el promedio de una lista de calificaciones y muestra cuántas están por encima del promedio.

```
calificaciones = [7.5, 8.0, 6.5, 9.0, 7.0, 8.5, 5.5]
# Tu código aquí
```

6.2. Ejercicios de while

Ejercicio 2.1: Suma acumulativa

Pide números al usuario hasta que introduzca 0. Muestra la suma total.

```
# Tu código aquí
```

Ejercicio 2.2: Validador de contraseña

Crea un programa que pida una contraseña hasta que sea correcta, con máximo 3 intentos.

```
contraseña_correcta = "python123"
# Tu código aquí
```

Ejercicio 2.3: Cuenta regresiva

Crea una cuenta regresiva desde 10 hasta 1, mostrando "¡Despegue!" al final.

```
# Tu código aquí
```

6.3. Ejercicios de for

Ejercicio 3.1: Tabla de multiplicar

Muestra la tabla de multiplicar del 1 al 10 para un número dado por el usuario.

```
# Tu código aquí
```

Ejercicio 3.2: Análisis de texto

Cuenta cuántas vocales hay en un texto dado.

```
texto = "Python es un lenguaje poderoso"
# Tu código aquí
```

Ejercicio 3.3: Lista de pares

Crea una lista con los números pares del 0 al 20.

```
# Tu código aquí
```

6.4. Ejercicios integradores

Ejercicio 4.1: Gestión de inventario

Crea un programa que gestione un inventario simple:

- Lista de productos con precios
- Calcular el precio total
- Encontrar el producto más caro

- Contar cuántos productos cuestan menos de 10€

```
# Tu código aquí
```

Ejercicio 4.2: Números primos

Encuentra todos los números primos entre 1 y 50.

```
# Tu código aquí
```

Ejercicio 4.3: Estadísticas de lista

Dada una lista de números, calcula:

- Media
- Mediana
- Mínimo y máximo
- Cuántos son pares e impares

```
numeros = [15, 23, 8, 42, 16, 4, 27, 33, 19, 50]
# Tu código aquí
```